

Software Requirements Specification (SRS)

AI Data Poisoning Mitigation System (Poison Guard)

Document Version: 1.0 Date: 2026-02-26 Status: Draft

Roll No	Name	Role
ASI23CA024	Avinash	Team Member
ASI23CA046	Nandhana Ramachandran	Team Member
ASI23CA030	Fayiza K H	Team Member
ASI23CA036	Jinendran S	Team Member
—	Dr Binju Saju	Project Mentor

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for the AI Data Poisoning Mitigation System (Poison Guard). The system detects potential poisoned samples in LLM training data and applies mitigation weights to reduce poisoning impact during training.

1.2 Scope

Poison Guard provides:

- Joint Influence Estimation (JIE) detection using TracIn-based influence scoring
- Representation-Level Outlier Detection (RLOD) using embedding-based outlier analysis
- Combined detection scoring for suspicious sample identification
- Mitigation weight generation for robust training
- API services for synchronous and asynchronous detection requests
- Integration pipeline support for multi-epoch adaptive scanning
- Frontend dashboard support for result visualization and monitoring

Primary security goal: reduce susceptibility to data poisoning attacks, particularly small-scale poisoning attempts (~250 samples).

1.3 Intended Audience

- Developers implementing detection and training components
- QA engineers validating detection quality and API behavior
- Project leads tracking deliverables and acceptance
- Operators deploying API and background workers

1.4 Definitions and Acronyms

JIE: Joint Influence Estimation

RLOD: Representation-Level Outlier Detection

TracIn: Training Data Influence tracing method

LLM: Large Language Model

FP Rate: False Positive Rate

Sync mode: API returns complete detection result in one response

Async mode: API returns job_id; result retrieved by polling

1.5 References

Project README

Project status and architecture documentation

API and deployment guides

2. Overall Description

2.1 Product Perspective

Poison Guard is a defense layer around model training workflows. It does not replace the target model; it augments training by detecting suspicious samples and modifying training influence through mitigation weights.

2.2 Product Functions

At minimum, the product shall:

1. Accept batches of training samples for poisoning detection.
2. Score samples with JIE and RLOD detectors.
3. Produce combined suspicion scores and mitigation weights.
4. Support both synchronous and asynchronous API workflows.
5. Provide health/readiness/job status endpoints.
6. Integrate with epoch-based adaptive sampling logic.
7. Persist or export result artifacts for analysis.

2.3 User Classes

ML Engineer: runs training and detection, consumes scores/weights.

API Consumer: sends detection requests and tracks jobs.

System Operator: deploys FastAPI, Celery/Redis, and monitoring.

Research Analyst: evaluates detection rates and false positives.

2.4 Operating Environment

OS: Windows/Linux (development), containerized Linux (deployment)

Language: Python 3.x

Frameworks: FastAPI, PyTorch

Model Stack: Hugging Face Transformers

Queue/Cache: Celery + Redis

Frontend: React + Vite (dashboard/chat/settings)

Optional acceleration: GPU for model inference/feature extraction

2.5 Constraints

Detection quality depends on checkpoint/model availability.

RLOD effectiveness depends on embedding quality and baseline fitting.

Sync mode latency bounded by configured timeout.

Async mode requires queue backend and worker availability.

Maximum samples/request enforced by configuration.

2.6 Assumptions and Dependencies

Valid model/tokenizer and checkpoint paths are configured.

Input samples follow expected JSON schema.

Redis and background worker are reachable in async deployments.

Poisoned/clean labeled datasets are available for validation testing.

3. External Interface Requirements

3.1 API Interface

3.1.1 Detection Endpoint: POST /api/detect

Input: samples (list), mode (sync|async)

Sync Output: status, latency_ms, results (scores/weights per sample)

Async Output: status=accepted, job_id

3.1.2 Job Status Endpoint: GET /api/jobs/{job_id}

Output: job state (processing/completed/failed/not_found) with progress/result when available.

3.1.3 Health and Readiness: GET /health, GET /ready

Output: service status objects for orchestration probes.

3.2 Data Interface

Configuration loaded from YAML file.

Result artifacts saved as JSON and/or PyTorch serialized files.

Embeddings and indexes cached in configured directories.

3.3 User Interface

Home view with product overview

Dashboard with metrics/charts/detection table

Chat interface for user prompts and responses

Settings for API and detection parameters

3.4 Hardware Interface

CPU execution must be supported as baseline.

GPU acceleration should be used when configured and available.

4. System Features and Functional Requirements

4.1 Sample Intake and Validation

FR-1 The system shall validate incoming requests before detection.

FR-2 The system shall reject requests exceeding configured limits.

FR-3 The system shall return descriptive errors for invalid inputs.

4.2 JIE Detection

FR-4 The system shall compute influence-based suspicion scores using configured checkpoints.

FR-5 The system shall support parameter-targeted gradient extraction as configured.

FR-6 The system shall return normalized per-sample JIE scores in range [0, 1].

4.3 RLOD Detection

FR-7 The system shall extract representation embeddings for candidate samples.

FR-8 The system shall fit/update outlier baseline from clean reference data.

FR-9 The system shall return per-sample RLOD scores.

4.4 Combined Scoring and Mitigation

FR-10 The system shall compute combined suspicion score using weighted JIE and RLOD contributions.

FR-11 The system shall compute mitigation weight inversely proportional to suspicion score.

FR-12 The system shall enforce minimum mitigation floor (0.1) and maximum cap (1.0).

4.5 Adaptive Epoch Scheduling

FR-13 The system shall support epoch-dependent sample scanning rate.

FR-14 The system shall allow aggressive scanning in early epochs and reduced in later epochs.

FR-15 The system shall expose per-epoch sampling summary for analysis.

4.6 Detection Execution Modes

FR-16 The system shall support synchronous detection for short-running requests.

FR-17 The system shall support asynchronous detection with returned job_id.

FR-18 The system shall support polling-based job status retrieval.

4.7 Result Persistence and Reporting

FR-19 The system shall persist detection outputs and summary metrics.

FR-20 The system shall include key statistics: detection rate, false positive rate, and average scores.

4.8 Training Pipeline Integration

FR-21 The system shall provide mitigation weights consumable by robust training routines.

FR-22 The system shall run multi-epoch detection/training cycles and aggregate results.

4.9 Security and Operational Controls

FR-23 The system shall support API authentication for protected deployments.

FR-24 The system shall support rate limiting to prevent abuse.

5. Non-Functional Requirements

5.1 Performance

NFR-1 Sync detection requests should complete within configured timeout budget.

NFR-2 Defense pipeline overhead target <25% additional training time.

NFR-3 Robust training overhead target <3% for core weighting path.

5.2 Reliability and Availability

NFR-4 API health/readiness probes shall accurately represent service state.

NFR-5 Async tasks shall survive transient API restarts with persistent queue backend.

5.3 Scalability

NFR-6 Async processing shall support concurrent jobs via worker scaling.

NFR-7 Request size limits shall protect service from overload.

5.4 Security

NFR-8 API endpoints shall support authenticated access in production.

NFR-9 Input validation shall prevent malformed payload processing.

NFR-10 Sensitive config values shall be externally configurable and not hardcoded.

5.5 Maintainability

NFR-11 Components shall remain modular (detectors, scheduler, API, training modules).

NFR-12 Core modules shall include unit/integration test coverage.

NFR-13 Documentation shall be maintained for setup, deployment, and usage.

5.6 Portability

NFR-14 System shall run in local Python environments and Docker-based deployments.

5.7 Observability

NFR-15 Service shall expose metrics and status endpoints for operations monitoring.

NFR-16 Detection pipeline shall log execution summaries for diagnostics.

6. Data Requirements

6.1 Input Sample Schema

Each sample should minimally include:

- sample_id (or equivalent unique ID)
- text or model input payload
- optional label (poisoned/clean) for evaluation runs

6.2 Output Score Schema

Each scored sample should include:

- sample_id
- jie_score
- rlod_score
- combined_score

mitigation_weight
optional classification label/status

6.3 Configuration Data

Configurable items include:

model/checkpoint locations
API ports, timeouts, max request samples
JIE and RLOD thresholds/parameters
training weight bounds and update cadence

7. Verification and Acceptance Criteria

7.1 Functional Acceptance

AC-1 Valid sync detection request returns scores and latency.
AC-2 Valid async detection request returns job_id, and polling returns terminal state.
AC-3 Combined scoring and mitigation weight values are returned for each processed sample.
AC-4 Health/readiness endpoints return successful status when services are operational.
AC-5 Adaptive scheduling changes scan rate according to epoch stage.

7.2 Quality Acceptance

AC-6 Detection quality meets project target thresholds in benchmark datasets:
Detection rate target: >85%
False positive target: <7%
AC-7 Pipeline overhead remains within accepted performance budget.

7.3 Deployment Acceptance

AC-8 Local run and containerized run complete with documented startup commands.
AC-9 Required dependencies (API, worker, cache) can be validated through readiness checks.

8. Risks and Mitigations

R-1: Incomplete RLOD implementation

Mitigation: Maintain integration contract and placeholder compatibility; complete detector module before final E2E validation.

R-2: Model/checkpoint path issues

Mitigation: Startup validation for config paths and explicit failure messages.

R-3: High sync latency under large batches

Mitigation: Route large workloads to async mode; enforce request limits.

R-4: False positives affecting clean data training

Mitigation: Threshold tuning, weighted mitigation floors, and benchmark-based calibration.

9. Future Enhancements (Out of Current Baseline Scope)

- Advanced attack scenario simulation expansion
- Dynamic threshold adaptation based on drift
- Explainability views for detector decisions
- Enhanced result export/reporting templates
- Multi-model support across additional LLM families

10. Requirements Traceability

Component	Requirements
JIE Module	FR-4, FR-5, FR-6
RLOD Module	FR-7, FR-8, FR-9
Combiner/Weighting	FR-10, FR-11, FR-12, FR-21
Adaptive Scheduler	FR-13, FR-14, FR-15
FastAPI/Celery APIs	FR-16, FR-17, FR-18, FR-23, FR-24
Integration Pipeline/Results	FR-19, FR-20, FR-22
Ops & Deployment	NFR-4, NFR-14, NFR-15

11. Sign-Off

This SRS is a living document and should be updated as module completion status changes (especially RLOD integration and full end-to-end validation milestones).